

Perfect 😊 Let's do the **actual analyzer architecture** now. This is where `Suspense` + streaming starts to feel practical rather than theoretical.

Lesson 4 — Streaming + Suspense in our Analyzer

Suppose our page is:

```
export default async function AnalyzePage() {
  const performance = await getPerformanceData();
  const analysis = await getAIAnalysis();

  return (
    <div>
      <PerformanceResults data={performance} />
      <AIResults data={analysis} />
    </div>
  );
}
```

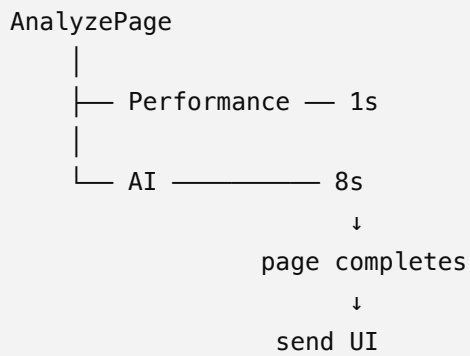
We already know the problem:

Performance API → 1 sec



Gemini AI → 8 sec

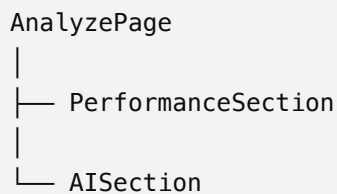
Because the page itself is waiting for both:



The user waits **8 seconds** before seeing anything useful.

1. Let's split the page into components

Instead, imagine:



And each section does its own data fetching.

PerformanceSection

```
async function PerformanceSection() {
  const data = await getPerformanceData();

  return <PerformanceResults data={data} />;
}
```

AISection

```
async function AISection() {
  const data = await getAIAnalysis();
```

```
return <AIResults data={data} />;  
}
```

Now our page can be:

```
export default function AnalyzePage() {  
  return (  
    <div>  
      <PerformanceSection />  
      <AISection />  
    </div>  
  );  
}
```

But there's a catch.

Both components are async Server Components.

So we need to tell React:

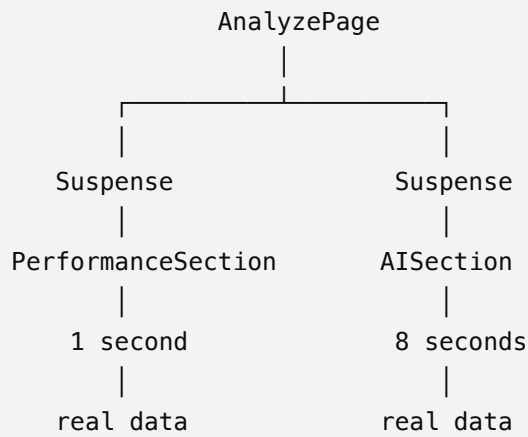
"It's okay if this component isn't ready yet. Show this fallback meanwhile."

That's exactly what `Suspense` does.

2. Add Suspense

```
import { Suspense } from "react";  
  
export default function AnalyzePage() {  
  return (  
    <div>  
      <Suspense fallback={<PerformanceSkeleton />}>  
        <PerformanceSection />  
      </Suspense>  
  
      <Suspense fallback={<AISkeleton />}>  
        <AISection />  
      </Suspense>  
    </div>  
  );  
}
```

Now look at what we've created:



3. What does the user experience?

Immediately

The user gets the page structure:

Website Performance
Loading performance...

AI Analysis
Generating analysis...

Then after 1 second:

Website Performance
Score: 92
LCP: 1.8s
CLS: 0.02

AI Analysis
Generating analysis...

Then after 8 seconds:

```
Website Performance
```

```
Score: 92
```

```
LCP: 1.8s
```

```
CLS: 0.02
```

```
AI Analysis
```

```
Your site performs well...
```

```
Here are 3 recommendations...
```

That's **streaming**.

The browser doesn't have to wait for the slowest piece.

4. Here's the key distinction

You might be thinking:

"Wait... didn't `loading.tsx` already show something while the page loads? Why do we need Suspense?"

Excellent question.

They're useful at **different levels**.

Imagine:

```
/analyze
```

```
├── Header
```

```
├── Performance
```

```
├── SEO
```

```
└── AI
```

With:

```
loading.tsx
```

you're essentially saying:

"The `/analyze` route is loading. Here's the loading UI for the route."

But with Suspense:

```
<Suspense fallback={<PerformanceSkeleton />}>
  <PerformanceSection />
</Suspense>
```

you're saying:

"Only THIS section is loading. The rest of the page can continue."

That's **fine-grained loading**.

5. Think of a restaurant 🍴

This analogy actually works ridiculously well.

You order:

- appetizer
- main course
- dessert

Bad experience

The waiter says:

"We'll bring everything when the dessert is ready."



So:

```
Appetizer → ready in 5 min
Main      → ready in 15 min
Dessert   → ready in 30 min
```

↓

Wait 30 min

↓

Bring everything



That's similar to making the whole page wait for the slowest operation.

Streaming experience

Instead:

```
5 min → appetizer arrives  
15 min → main course arrives  
30 min → dessert arrives
```



That's streaming.

And `Suspense` is essentially saying:

"While this particular dish isn't ready, put this placeholder here."

6. One subtle but VERY important thing

Let's compare these two approaches.

Approach A — fetch everything in the page

```
export default async function AnalyzePage() {  
  const performance = await getPerformanceData();  
  const ai = await getAIAnalysis();  
  
  return (...);  
}
```

The page is blocked by the slowest operation.

```
Performance —┐  
                └─→ Page waits  
AI —————┐
```

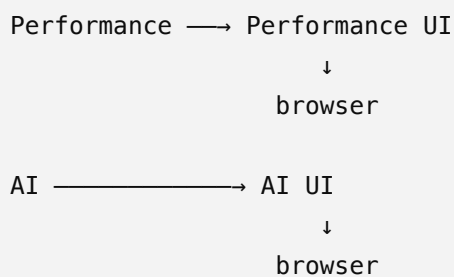


Approach B — let sections fetch independently

```
export default function AnalyzePage() {
  return (
    <>
      <Suspense fallback={<PerformanceSkeleton />}>
        <PerformanceSection />
      </Suspense>

      <Suspense fallback={<AISkeleton />}>
        <AISection />
      </Suspense>
    </>
  );
}
```

Now:



Each boundary can resolve independently.

7. But what about parallel fetching?

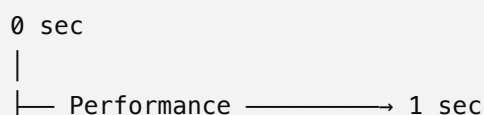
Here's a connection to what we learned earlier.

We learned:

```
const [performance, analysis] = await Promise.all([
  getPerformanceData(),
  getAIAnalysis(),
]);
```

That solves **sequential vs parallel fetching**.

Both APIs start together:



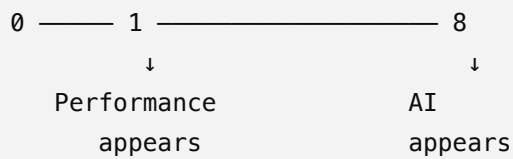


That's good.

But the page still waits until **both** finish:



Suspense + streaming solves a **different problem**:



So:

`Promise.all`

Start independent requests together.

`Suspense`

Don't make one slow component block another component's UI.

Streaming

Send completed UI to the browser progressively.



That's an important interview-level distinction.

8. What about `loading.tsx` then?

Our final analyzer could have:

```

app/
├── analyze/
│   ├── page.tsx
│   ├── loading.tsx
│   └── error.tsx
  
```

And inside `page.tsx` :

```
<Suspense fallback={<PerformanceSkeleton />}>
  <PerformanceSection />
</Suspense>

<Suspense fallback={<AISkeleton />}>
  <AISection />
</Suspense>
```

So we can have **both**.

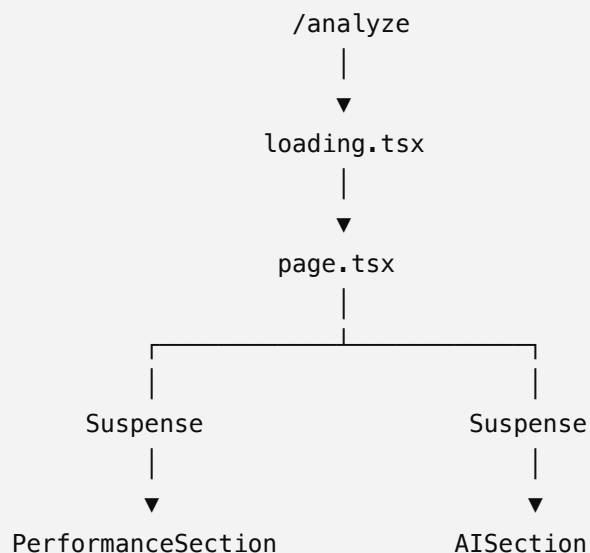
Conceptually:

```
/analyze
├── loading.tsx
│   └── ↓
│       Route-level loading
└── page.tsx
    ├── Suspense
    │   └── Performance
    └── Suspense
        └── AI
```

This gives us multiple levels of loading UX.

9. The architecture we're building

Our analyzer is now starting to look like a real application:





And if something unexpected happens:

Google API → ✨ → error.tsx
 Gemini API → ✨ → error.tsx



If the requested resource doesn't exist:

/analyze/some-invalid-id
 ↓
 notFound()



So we're actually putting **all the Lesson 4 concepts together now.**

One last mental model 🧠

I want you to remember this chain:

User requests route
 ↓
 loading.tsx
 "Something is loading"
 ↓
 page starts rendering
 ↓
 Suspense boundaries
 "These individual pieces can load independently"
 ↓
 Streaming
 "Send each ready piece as soon as it's ready"
 ↓
 Final UI



And errors have their own path:

Expected invalid input
 ↓
 handle it normally

 Resource doesn't exist



```
↓  
notFound()  
  
Unexpected failure  
↓  
error.tsx
```

Next: Final Lesson 4 Architecture Exercise

Now I want **you** to design the analyzer architecture.

I'll give you a scenario and you tell me where each concept belongs:

A user enters a URL and clicks **Analyze**.

The app needs:

1. Google PageSpeed data
2. AI analysis using Gemini
3. A results page with Performance + AI sections
4. Performance can finish much earlier than AI
5. Invalid/nonexistent analysis IDs should show a 404
6. Unexpected API failures shouldn't crash the whole application
7. The user should see useful loading UI immediately

Tell me what you'd use for each of these:

1. Route-level loading → ?
2. Individual section loading → ?
3. Progressive delivery → ?
4. Nonexistent resource → ?
5. Unexpected failure → ?
6. Parallel independent API calls → ?



Don't worry about getting it perfect. **This is the exercise that tells me whether Lesson 4 has actually clicked.** 😊



what exactly are async Server Components.?? and also i m confused see whether its sequential means wait for long and parallel means wait for slower req time..we are

waiting so both show suspense and show results at the same time right where does streaming come here then

Yes — **your confusion is exactly at the right spot**. The missing piece is that **parallel fetching and streaming solve two different problems**.

Let's slow this down and separate them. 📌

1. First: what is an async Server Component?

You already know a normal Server Component:

</> TypeScript

```
export default function Page() {  
  return <h1>Hello</h1>;  
}
```



It runs on the **server** by default in Next.js App Router.

Now make it **async** :

</> TypeScript

```
export default async function Page() {  
  const data = await fetch("https://api.example.com/data");  
  
  return <h1>{data.title}</h1>;  
}
```



This is an **async Server Component**.

It simply means:

A Server Component that can pause while it waits for asynchronous work, usually data fetching, before producing its UI.

So:

Server Component

+

async

↓

Can await data

↓

Then produce UI



And importantly, `async` does **not** mean "runs asynchronously in the browser."

It means the **server-side component function can use** `await`.

2. Now your really good question: sequential vs parallel

Suppose:

Performance API = 2 sec
Gemini API = 8 sec



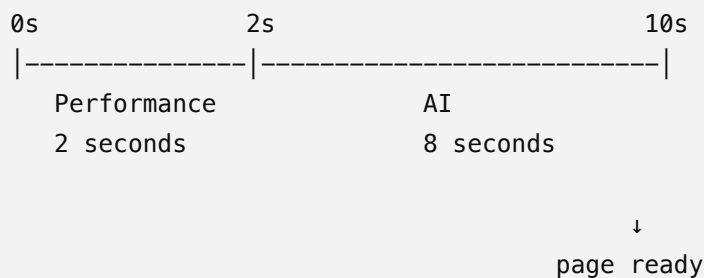
Sequential

`</>` TypeScript



```
const performance = await getPerformance();  
const ai = await getAI();
```

Timeline:



Total:

2 + 8 = 10 seconds

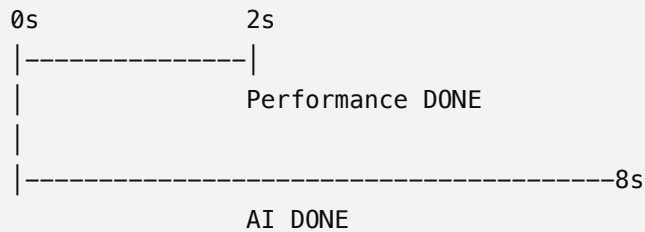
Parallel

`</>` TypeScript



```
const [performance, ai] = await Promise.all([  
  getPerformance(),  
  getAI(),  
]);
```

Timeline:



Total:

$\max(2, 8) = 8 \text{ seconds}$

So you are absolutely right:

Parallel still waits for the slower request if the page needs both results before it can render the page.

That's the key!

3. Then where the heck does Streaming come in? 😊

This is the exact distinction.

With parallel fetching:

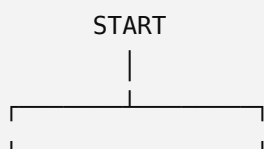
`</> TypeScript`

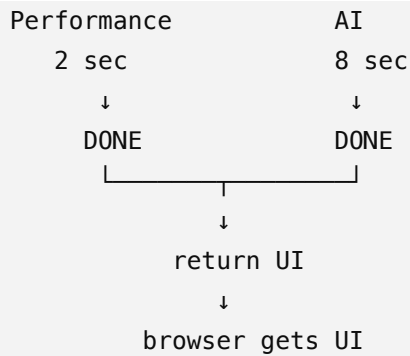
```
const [performance, ai] = await Promise.all([
  getPerformance(),
  getAI(),
]);

return (
  <>
    <Performance data={performance} />
    <AI data={ai} />
  </>
);
```

Even though both requests started together...

the `return` doesn't happen until BOTH are finished.





So the browser doesn't get the Performance UI at 2 seconds.

It gets **everything at 8 seconds**.

4. Streaming changes that

Instead of making the **whole page** wait for both:

`</>` TypeScript

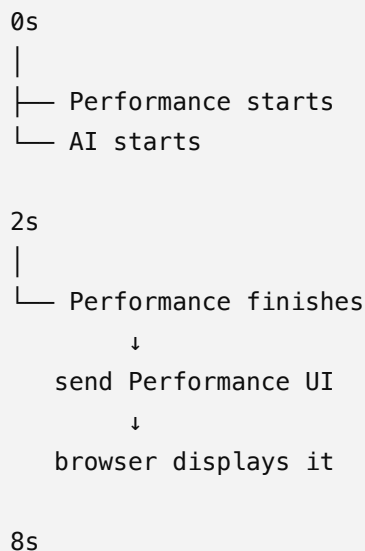
```
<Suspense fallback={<PerformanceSkeleton />}>
  <PerformanceSection />
</Suspense>

<Suspense fallback={<AISkeleton />}>
  <AISection />
</Suspense>
```

Now we're telling React:

"These are separate pieces. If one is ready, don't make it wait for the other."

So:



```
└─ AI finishes  
    ↓  
  send AI UI  
    ↓  
browser displays it
```

THAT is where streaming happens.

5. So your sentence needs one tiny correction

You said:

"we are waiting so both show suspense and show results at the same time right"

Not quite.

With Suspense:

They both START around the same time.

But they **don't** necessarily show their results at the same time.

Instead:

Performance → finishes at 2s → appears at 2s
AI → finishes at 8s → appears at 8s



That's the whole point.

6. Here's the easiest comparison

Imagine you order two things:

🍕 Pizza → 2 minutes
🍰 Cake → 8 minutes



Sequential

You say:

"Don't even start the cake until the pizza is done."

Pizza: █████

Cake: ██████████

Total = 10 min



That's sequential fetching.

Parallel, but no streaming

You tell the kitchen:

"Start both!"

Pizza: █████

Cake: ██████████



Pizza is ready at 2 minutes.

But you tell the waiter:

"Don't bring anything until BOTH are ready."

So:

2 min → Pizza ready 🍕



waits...



8 min → Cake ready 🍰



bring both together



That's **parallel fetching without streaming**.

Parallel + Suspense + Streaming

You say:

"Bring each one as soon as it's ready."

2 min → 🍕 Pizza arrives

8 min → 🍰 Cake arrives



That's **streaming**.

🔥 Now you can see the difference.

7. So what does Suspense actually do?

Suspense creates a boundary around a piece of UI:

⌄ TypeScript

```
<Suspense fallback=<AISkeleton />>
  <AISection />
</Suspense>
```



You're basically saying:

"If `AISection` isn't ready yet, this part of the UI can be represented by `AISkeleton`."

So at first:

Performance
Loading...

AI
Loading...



Then Performance resolves:

Performance
Score: 92

AI
Loading...



Then AI resolves:



```
Performance
Score: 92
```

```
AI
Here's your analysis
```

That progressive replacement is what makes streaming useful.

8. One VERY important architecture detail

There's a subtlety I want you to understand because otherwise this topic gets muddy.

If you do this:

 TypeScript

```
export default async function Page() {
  const performance = await getPerformance();
  const ai = await getAI();

  return (
    <>
      <Suspense>...</Suspense>
    </>
  );
}
```

You've already waited for the data **before reaching the Suspense boundaries**.

So you've defeated much of the benefit.

Instead, you want the components themselves to own the waiting:

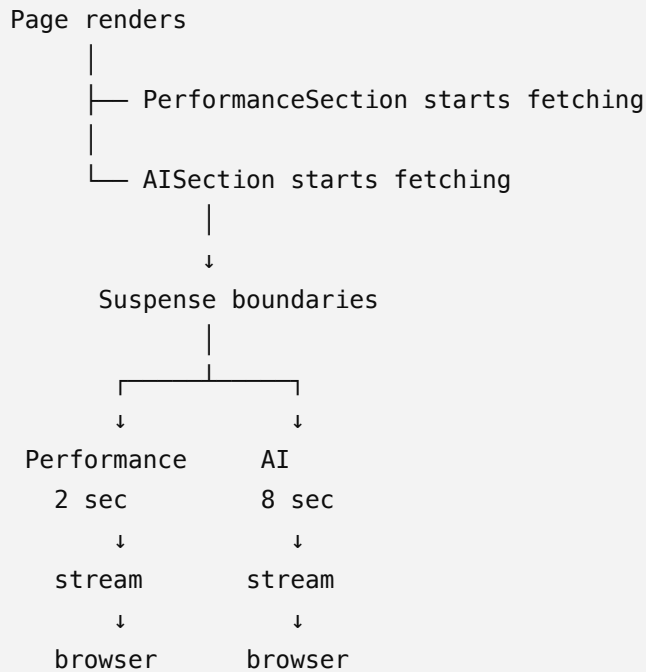
 TypeScript

```
export default function Page() {
  return (
    <>
      <Suspense fallback={<PerformanceSkeleton />}>
        <PerformanceSection />
      </Suspense>

      <Suspense fallback={<AISkeleton />}>
        <AISection />
      </Suspense>
    </>
  );
}
```

```
    </Suspense>
  </>
);
}
```

Then:



The four concepts — finally separated

Concept	What problem does it solve?
Sequential fetching	Requests depend on each other / but can accidentally make things slower
Parallel fetching	Start independent requests together
Suspense	Let individual UI sections have their own loading boundary
Streaming	Deliver ready UI to the browser without waiting for the entire page

And `loading.tsx` is basically:

A convenient route-level loading boundary provided by Next.js.

The one sentence I want you to remember

Parallel determines WHEN requests start; Suspense determines WHERE loading boundaries are; streaming determines WHEN the resulting UI can reach the browser.

That's the missing puzzle piece. 🧩

And honestly, **once that sentence clicks, this whole section becomes much easier.**



Yes — **you've caught the exact subtle point.** There are actually **two different things getting mixed together** here:

1. How a Server Component fetches data
2. How Next.js/React renders and sends different parts of the UI

Let's untangle them carefully. 🧩

1. Are Server Components normally synchronous?

Yes, syntactically they usually are.

For example:

```
</> TypeScript

export default function Page() {
  return <h1>Hello</h1>;
}
```



That's a synchronous Server Component.

But Server Components **can also be async:**

```
</> TypeScript
```



```
export default async function Page() {  
  const data = await getData();  
  
  return <h1>{data.title}</h1>;  
}
```

So:

```
Server Component  
├── function Page()      ← sync  
└── async function Page() ← async Server Component
```

The important thing is:

"Server Component" and "async component" are not opposites.

Server Component describes **where the component executes**.

`async` describes **whether the component function can await asynchronous work**.

So most simple Server Components may be sync, but a Server Component that directly awaits data can be async.

2. Now your bigger question: Promise.all

You are absolutely right:

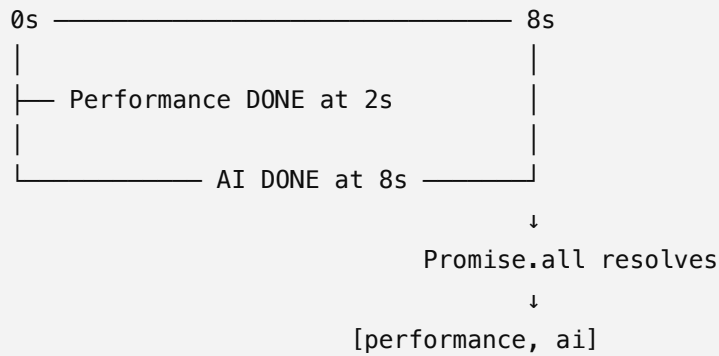
```
</> TypeScript  
  
const [performance, ai] = await Promise.all([  
  getPerformance(),  
  getAI(),  
]);
```

`Promise.all()` gives you the combined result **only after ALL promises resolve**.

Suppose:

```
Performance = 2 sec  
AI           = 8 sec
```

Then:



So if your page does:

```

</> TypeScript

const [performance, ai] = await Promise.all(...);

return (
  <>
    <Performance data={performance} />
    <AI data={ai} />
  </>
);
  
```

There is **NO** streaming of Performance at 2 seconds.

You are correct.

It can't stream that completed Performance result because the component hasn't even reached its `return` yet.

3. So how CAN streaming happen?

Here's the key:

We don't use `Promise.all()` in the parent for those two pieces.

Instead, we do:

```

</> TypeScript

export default function Page() {
  return (
    <>
      <Suspense fallback={<PerformanceSkeleton />}>
        <PerformanceSection />
      </Suspense>

      <Suspense fallback={<AISkeleton />}>
  
```

```
    <AISection />
  </Suspense>
</>
);
}
```

Notice something VERY important:

Page itself isn't waiting for either API.

It can immediately produce this structure:

```
Page
├── Suspense
│   └── PerformanceSection
└── Suspense
    └── AISection
```

Then React can render those async Server Components separately.

4. What happens now?

At time 0

Page says:

Here's my UI:

Performance area → Suspense

AI area → Suspense

The boundaries have fallbacks:

```
┌ Performance
└ Loading...

┌ AI
└ Loading...
```

The server starts resolving:

PerformanceSection → fetching...
AISection → fetching...



At 2 seconds

Performance finishes.

React now has:

PerformanceSection → READY ✓
AISection → STILL WAITING ⌚



So it can produce:

Performance
Score: 92

AI
Loading...



And **that completed portion can be streamed to the browser.**

At 8 seconds

AI finishes:

PerformanceSection → READY ✓
AISection → READY ✓



Now the second portion can be streamed:

AI
Your site performs...



5. So here's the crucial difference

You were thinking:

"Parallel fetching means Promise.all, and Promise.all waits for both, so how can streaming happen?"

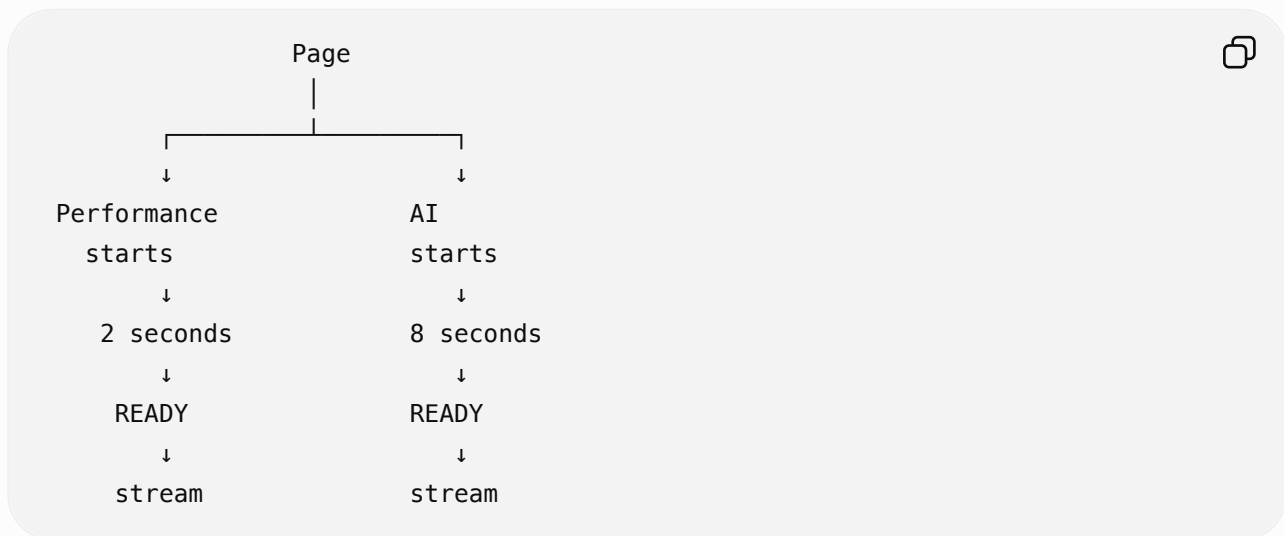
The answer is:

Parallel fetching does NOT necessarily mean `Promise.all`.

`Promise.all()` is one way of coordinating parallel promises.

You can have parallel work without waiting for a combined result.

For example:



They're both running concurrently.

But we're **not creating one giant promise that the parent must await before it can render.**

6. This is the distinction you were missing

`Promise.all`

Means:

"Start these together, and **give me their combined results when ALL are finished.**"

`</>` TypeScript

```
const results = await Promise.all([...]);
```

Therefore:

2 sec → Performance ready



WAIT



8 sec → AI ready



Combined result



Suspense + async Server Components

Means:

"These pieces can independently suspend while they're waiting. Give me whatever part is ready."

⟨⟩ TypeScript



```
<Suspense>
  <PerformanceSection />
</Suspense>
```

```
<Suspense>
  <AISection />
</Suspense>
```

Therefore:

2 sec → Performance ready



STREAM IT

8 sec → AI ready



STREAM IT



7. One analogy that should make this click

Imagine two people are making your food:

👤 Person A → Pizza → 2 min

👤 Person B → Cake → 8 min



`Promise.all`

You tell the waiter:

"Don't bring me anything until BOTH people finish."

So:

Pizza ready → waits
Cake ready → both arrive



Suspense + Streaming

You tell the waiter:

"Bring each dish whenever it's ready."

So:

2 min → 🍕 arrives
8 min → 🍰 arrives



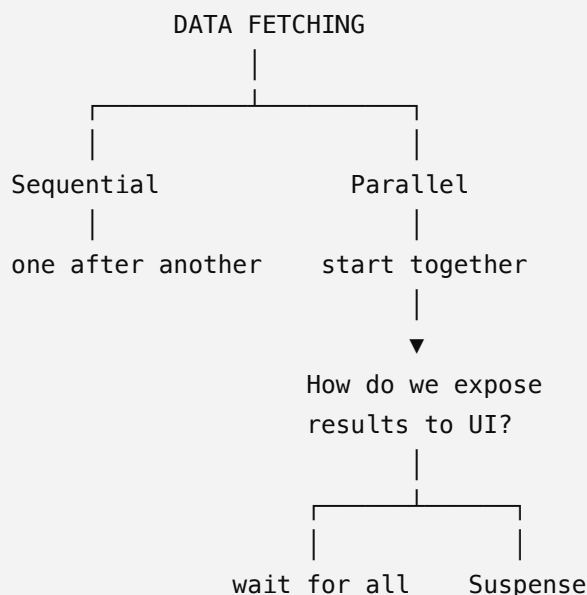
Both cooks were working in parallel.

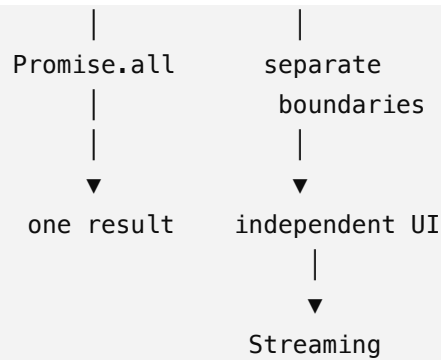
But you didn't force the waiter to wait for both.

That's streaming.

8. And this is why I want you to separate these ideas

Think of three layers:





That's the architecture-level distinction.

And one correction to something I said earlier

I said:

"Parallel determines WHEN requests start; Suspense determines WHERE loading boundaries are; streaming determines WHEN resulting UI can reach the browser."

That's a good mental model, **but don't equate parallel fetching with `Promise.all()`**.

`Promise.all()` is specifically:

parallel execution + wait for all results as one combined result.

Streaming requires that we **don't make the parent wait for that combined result**.

 **So your current understanding should be:**

Server Components

↓

can be sync OR async

Parallel fetching

↓

multiple requests can run at the same time

`Promise.all()`

↓

wait for ALL of those parallel requests

↓

combined result

Suspense

↓

allow individual async UI pieces to wait independently



Streaming



send each completed piece to browser
without waiting for slower pieces



That's it. You've actually found the important nuance rather than just memorizing "Suspense = loading." 😊

The next thing worth understanding is **exactly what React is doing when an async Server Component "suspends" and how the server can send HTML/RSC chunks progressively.** That's the final conceptual piece before the architecture exercise.



how to run multiple requests at the same time without using promise.all..also how sequential and suspense and streaming works??

Yes. Let's make this **very concrete**, because there are actually **two separate questions** hiding here:

1. How do requests run concurrently without `Promise.all`?
2. How does that connect to Suspense + streaming?

1. Multiple requests can run at the same time without `Promise.all`

This is the first key idea:

`Promise.all()` does NOT make requests parallel.

The requests become concurrent because you **start them before awaiting them.**

For example:

`</>` TypeScript



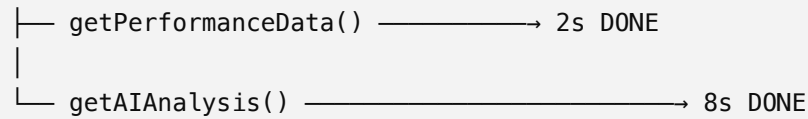
```
const performancePromise = getPerformanceData();  
const aiPromise = getAIAnalysis();
```

At this point, both have started.

time →



0s



No `Promise.all()` was needed to start them concurrently.

`Promise.all()` is just a convenient way to say:

"Now wait until all these already-running promises finish and give me their results together."

For example:

`</> TypeScript`



```
const performancePromise = getPerformanceData();
const aiPromise = getAIAnalysis();

// both are already running

const performance = await performancePromise;
const ai = await aiPromise;
```

Both requests started at the top.

But notice something subtle:

Performance finishes → 2 sec
AI finishes → 8 sec



If we eventually need **both results**, we still can't produce the complete combined UI until 8 seconds.

2. Sequential fetching

Now compare that with:

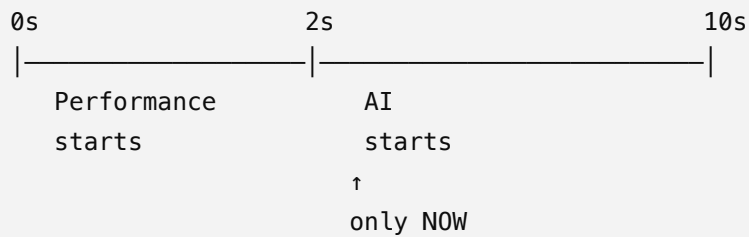
`</> TypeScript`



```
const performance = await getPerformanceData();
```

```
const ai = await getAIAnalysis();
```

Here the second request doesn't even start until the first one finishes.



So:

Sequential:

```
Request A
  ↓
wait
  ↓
Request B
  ↓
wait
  ↓
result
```

Total:

2 + 8 = 10 seconds

3. Parallel/concurrent fetching

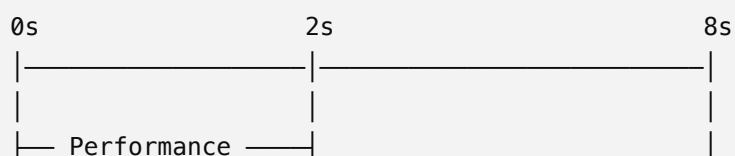
Without `Promise.all`:

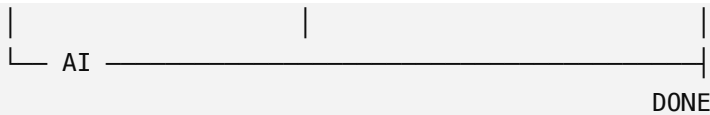
</> TypeScript

```
const performancePromise = getPerformanceData();
const aiPromise = getAIAnalysis();

const performance = await performancePromise;
const ai = await aiPromise;
```

Both start immediately:





Total time is about **8 seconds**, not 10.

So far so good.

But...

4. Here's where your question about Streaming becomes important

You might now say:

"Okay, Performance finished at 2 seconds. Why can't I send it to the browser?"

Because **your particular code is still waiting for both results before returning the page.**

</> TypeScript



```
const performancePromise = getPerformanceData();
const aiPromise = getAIAnalysis();

const performance = await performancePromise;
const ai = await aiPromise;

return (
  <>
    <Performance data={performance} />
    <AI data={ai} />
  </>
);
```

Look at the order:

Start both



```
↓
await Performance
↓
Performance ready at 2s
↓
await AI
↓
AI ready at 8s
↓
return page
```

Therefore:

```
2 sec → Performance is ready
      ↓
    BUT WAIT
      ↓
8 sec → AI ready
      ↓
    return entire UI
```



No useful streaming of Performance happens here.

And this is exactly what you were noticing earlier. 👍

5. So how does Suspense change it?

Instead of the parent doing this:

</> TypeScript

```
const performance = await getPerformanceData();
const ai = await getAIAnalysis();
```



we let the **individual Server Components** do their own waiting.

</> TypeScript

```
export default function Page() {
  return (
    <>
      <Suspense fallback={<PerformanceSkeleton />}>
        <PerformanceSection />
      </Suspense>

      <Suspense fallback={<AISkeleton />}>
        <AISection />
      </Suspense>
    </>
  );
}
```



And:

</> TypeScript

```
async function PerformanceSection() {
  const data = await getPerformanceData();
```



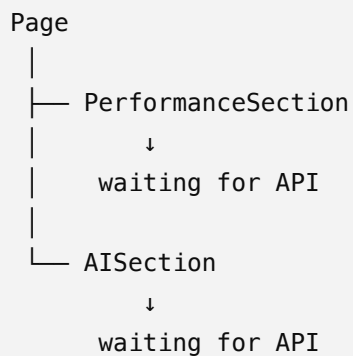
```
return <PerformanceResults data={data} />;  
}
```

</> TypeScript

```
async function AISection() {  
  const data = await getAIAnalysis();  
  
  return <AIResults data={data} />;  
}
```

Now the parent **doesn't await** either request.

Instead:



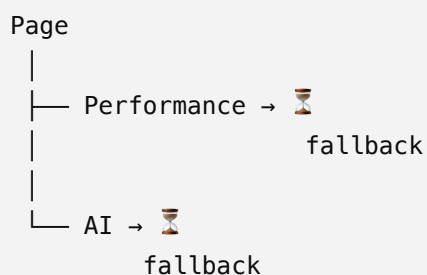
Both sections can be resolving independently.

6. Now we can finally see Streaming

Let's say:

```
Performance = 2 sec  
AI           = 8 sec
```

At 0s :



Browser can get something like:

Performance
Loading...

AI
Loading...



Then at **2 seconds**:

Performance → 
AI → 



The server can stream the newly resolved Performance content:

Performance
Score: 92

AI
Loading...



Then at **8 seconds**:

Performance → 
AI → 



The AI content gets streamed:

Performance
Score: 92

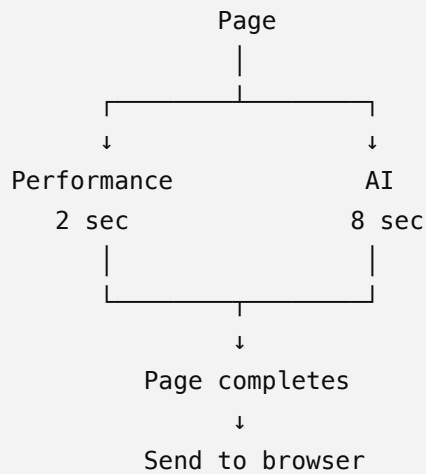
AI
Your website...



7. So what exactly is happening?

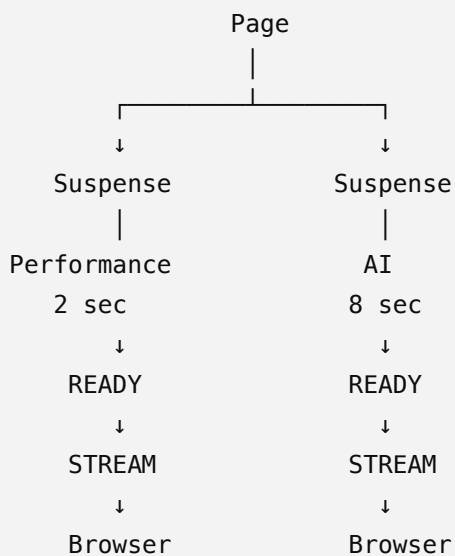
This is the part I want you to really visualize.

Without Suspense



Everything effectively gets held behind the slowest operation.

With Suspense



That's the difference.

8. And now sequential vs parallel vs Suspense vs streaming

Here's the cleanest mental model.

Sequential

</> TypeScript



```
const a = await A();  
const b = await B();
```

Means:

Don't start B until A finishes.

A → B →



Parallel

</> TypeScript



```
const aPromise = A();  
const bPromise = B();
```

Means:

Start A and B without making B wait for A.

A →
B →



You can use:

</> TypeScript



```
await Promise.all([A(), B()]);
```

if you want their results together.

Suspense

</> TypeScript



```
<Suspense fallback={<Skeleton />}>  
  <SomeAsyncComponent />  
</Suspense>
```

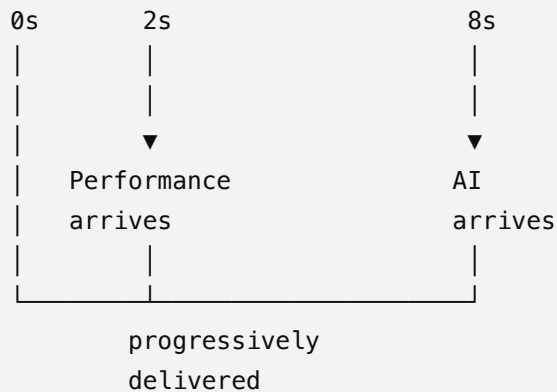
Means:

This part of the UI is allowed to wait independently.

Streaming

Means:

As those independently suspended parts become ready, their UI can be sent to the browser progressively.



9. One more thing: "without Promise.all" doesn't automatically mean streaming

This is really important.

This:

`</> TypeScript`

```
const aPromise = A();
const bPromise = B();

const a = await aPromise;
const b = await bPromise;
```

is parallel fetching.

It is **NOT** automatically streaming.

Why?

Because your component still waits until it has whatever it needs before returning.

Streaming requires a rendering structure where parts of the UI can resolve independently — that's where **Suspense boundaries** come in.

So:

Parallel

↓

Requests run concurrently

Suspense

↓

UI pieces can wait independently

Streaming

↓

Those resolved pieces can reach browser progressively



10. And there's an important correction to your mental model

Don't think:

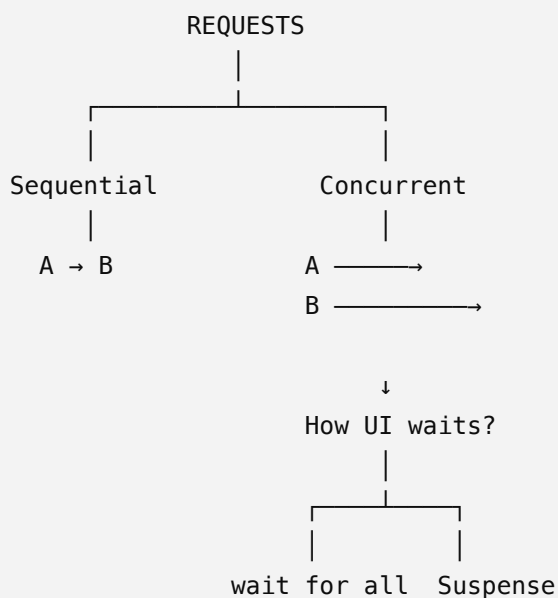
`Promise.all` → parallel

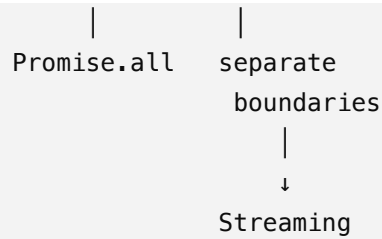
and

`Suspense` → parallel

They're not the same thing.

Think:





The killer sentence:

Parallel fetching is about when the requests run. Suspense is about how the UI waits. Streaming is about when the UI gets delivered.

That distinction is the whole game. 🎯

And **yes**, your earlier objection about `Promise.all` was completely valid: if the parent awaits `Promise.all`, the parent cannot stream the first completed result from that `Promise.all` because it doesn't receive the combined result until the slowest promise finishes.



but say i have req A and req B depends on req A result so its ideally sequential but i dont need to show both results at once i can show completed one first and next after its done so its sequential req but if i need suspense and streaming how to do

YES! 🔥 This is the exact case that makes the distinction crystal clear.

You have:

```
graph TD; A[Request A] --> B[A's result is needed to start B]; B --> C[Request B];
```



So the requests **MUST** be sequential.

But you are saying:

"I don't need to wait to show A's UI. As soon as A finishes, show A. Then start B, and

when B finishes, show B."

Absolutely. You can do that. And this is a case where **sequential fetching + Suspense + streaming** can all coexist.

1. Your scenario

Suppose:

A = Get website information
B = Get PageSpeed data for that website

B needs A:

A: find website/project
↓
result
↓
B: use A's result to call PageSpeed

For example:

A → 2 sec
B → 5 sec

You want:

```
0s
|
|— A starts
|
2s
|
|— A finishes
|
|— Show A UI immediately ✅
|
|— Start B
|
7s
|
|— B finishes
|
|— Show B UI
```

That's perfectly reasonable.

2. The mistake would be putting both awaits in the parent

If you do:

❏ TypeScript

```
export default async function Page() {  
  const a = await getA();  
  
  const b = await getB(a);  
  
  return (  
    <>  
    <AResult data={a} />  
    <BResult data={b} />  
    </>  
  );  
}
```

Then:

A finishes → 2 sec
↓
B starts
↓
B finishes → 7 sec
↓
return
↓
browser gets A + B

So even though **A was ready at 2 seconds**, you're holding its UI until 7 seconds.

That's the thing you don't want.

3. So where should the Suspense boundary go?

You want **A** to be its own independently renderable piece.

For example:

❏ TypeScript

```
export default function Page() {
  return (
    <>
      <Suspense fallback={<ALoading />}>
        <ASection />
      </Suspense>

      <Suspense fallback={<BLoading />}>
        <BSection />
      </Suspense>
    </>
  );
}
```

But there's a problem.

How does `BSection` get A's result?

Because:

B depends on A



We can't simply have both independently fetch:

A —————→
B —————→



because that would make B start before A finishes.

So we need to **pass A's result to B**.

4. One way: nest the components

This is the interesting part.

```
</> TypeScript

export default function Page() {
  return (
    <Suspense fallback={<ALoading />}>
      <ASection />
    </Suspense>
  );
}
```



Then:

</> TypeScript



```

async function ASection() {
  const a = await getA();

  return (
    <>
      <AResult data={a} />

      <Suspense fallback=<BLoading />>
        <BSection a={a} />
      </Suspense>
    </>
  );
}

```

And:

</> TypeScript



```

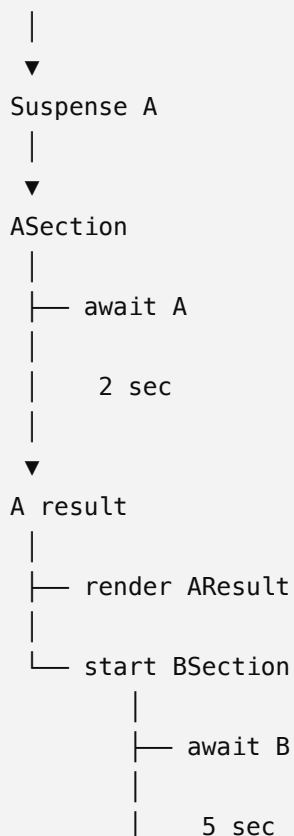
async function BSection({ a }) {
  const b = await getB(a);

  return <BResult data={b} />;
}

```

Now look at the execution:

Page



↓
B result

5. Now streaming can happen

At the beginning:

Browser:



Loading A...

After A finishes:

Browser:



A Result
Website: example.com

Loading B...

Then B finishes:

Browser:



A Result
Website: example.com

B Result
Performance: 92

So:

Requests

A → B



Sequential.

UI

A UI → B UI



Progressively rendered.

Delivery

A UI → streamed
B UI → streamed later



Streaming.



That's the important insight.

6. So Suspense does NOT mean requests must be parallel

This is another misconception worth killing.

Suspense doesn't say:

"Make everything parallel."

It says:

"If this part isn't ready, allow this boundary to show its fallback instead of blocking the UI around it."

Therefore:

Sequential requests
+
Suspense boundaries
+
Streaming



is completely valid.

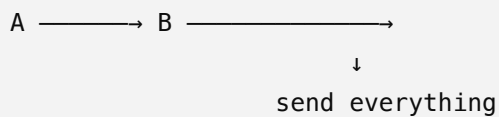
7. Let's compare the two architectures

❌ Sequential + wait for everything

</> TypeScript

```
async function Page() {  
  const a = await A();  
  const b = await B(a);  
  
  return (  
    <>  
    <AResult data={a} />  
    <BResult data={b} />  
    </>  
  );  
}
```

Timeline:



User sees:

2 sec → nothing useful
7 sec → A + B

✅ Sequential + streaming

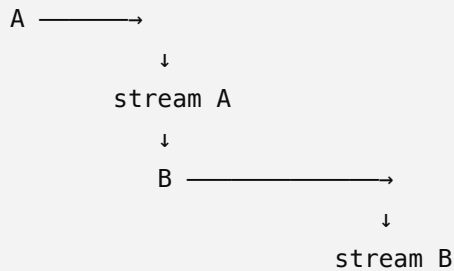
</> TypeScript

```
function Page() {  
  return (  
    <Suspense fallback={<ALoading />}>  
      <ASection />  
    </Suspense>  
  );  
}  
  
async function ASection() {  
  const a = await A();  
  
  return (  
    <>
```

```
<AResult data={a} />

<Suspense fallback={<BLoading />}>
  <BSection a={a} />
</Suspense>
</>
);
}
```

Timeline:



User sees:

2 sec → A
7 sec → B

That's exactly what you wanted.

8. And this reveals something REALLY important

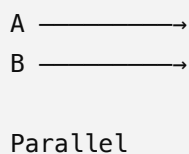
There are actually two independent dimensions:

Dimension 1 — Data dependencies

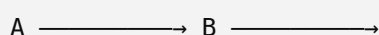
Ask:

Can B start without A?

If yes:



If no:



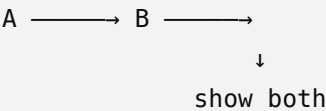
Sequential

Dimension 2 — UI delivery

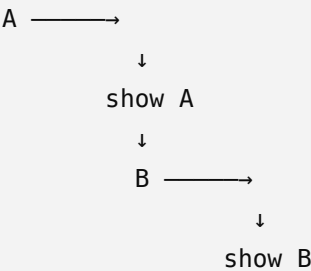
Ask:

Do I need to wait for everything before showing anything?

If yes:



If no:



And **Suspense + streaming** gives you the second behavior.

🧠 So remember this matrix

	Wait for whole UI	Show pieces progressively
Parallel requests	<code>Promise.all</code> → render everything	Suspense + streaming
Sequential requests	<code>await A</code> → <code>await B</code> → render everything	Suspense + streaming ✅

That last box is **exactly your example**.

So you don't choose:

"Sequential OR Suspense"

or:

"Parallel OR Streaming"

They're solving **different dimensions of the problem.**

Sequential/parallel = data dependency.

Suspense/streaming = UI rendering and delivery.

And THAT is the conceptual bridge I wanted you to reach. 🎯



ChatGPT can make mistakes. Check important info.